

邏輯設計 期末專題報告書

循序電路設計 自動化系統

Sequential Circuit Design Automation System

學號 1140503

姓名 張昱謙

中華民國 115 年 6 月

線上展示：<https://doremi0614.github.io/logic/>

目錄

- 一、專題簡介與動機
- 二、系統功能總覽
- 三、理論背景
- 四、完整製作流程
- 五、系統架構與技術細節
- 六、操作說明與實作範例
- 七、成果展示
- 八、開發過程遭遇的問題與解決
- 九、結論與心得
- 十、參考資料

一、專題簡介與動機

1.1 專題動機

在數位邏輯設計課程中，「循序電路 (Sequential Circuit) 」的設計流程相當繁瑣：從狀態表 (State Table) 開始，需要依序完成狀態指定、查激勵表、填卡諾圖、化簡布林方程式，最後才能畫出電路圖。整個過程不僅費時，人工化簡時也很容易出錯。本專題的動機，便是希望把這一連串「機械化」的設計步驟自動化，讓使用者只要輸入狀態表，系統就能即時算出結果並視覺化呈現，作為學習與驗算的輔助工具。

1.2 專題目標

- 以網頁形式 (單頁式應用程式 SPA) 實作，免安裝、跨平台、可直接於瀏覽器使用。
- 支援 Mealy 與 Moore 兩種模型，以及 JK、T、D 三種正反器。
- 由狀態表自動推導：狀態指定 → 激勵表 → 卡諾圖化簡 → 正反器輸入方程式。
- 將結果視覺化：即時卡諾圖、自動佈線的電路圖、狀態轉換泡泡圖。
- 可匯出設計報告，並部署到網際網路供任何人使用。

1.3 系統概觀

系統採用現代、乾淨的淺色系工程軟體風格，畫面分為三個直行：左欄為輸入區 (淡紅邊框)、中欄為運算結果區 (淡藍邊框)、右欄為電路圖區 (淡綠邊框)，最下方為操作列。整體具備響應式設計 (RWD)，在較小螢幕時會自動由三欄變為單欄垂直排列。

Sequential Circuit Design Automation System

1. MODEL TYPE

Mealy Model
Output depends on present state and input

Moore Model
Output depends on present state only

2. FLIP-FLOP TYPE

JK Flip-Flop
Excitation inputs J / K per state variable

T Flip-Flop
Single toggle input T per state variable

D Flip-Flop
Single data input D = next-state value

3. STATE TABLE INPUT

Input Variables

X

Output Variables

Z

PRESENT STATE	X	NEXT STATE	Z
A	0	A	0
A	1	B	0
B	0	C	0
B	1	B	1
C	0	A	1
C	1	B	0

+ Add Row
Clear Table
Load Example

STATE TRANSITION DIAGRAM

Edge label: X/Z - updates live with the table

OUTPUT 1: FLIP-FLOP INPUT EQUATIONS

State Variables: Q1 Q0

Flip-Flop Input Equations (Simplified)

FLIP-FLOP	INPUT	EQUATION
FF1	J1	$J1 = Q0 \cdot \bar{X}$
FF1	K1	$K1 = 1$
FF0	J0	$J0 = X$
FF0	K0	$K0 = \bar{X}$

Output equation: $Z = Q0 \cdot X + Q1 \cdot \bar{X}$ (Mealy model)

K-MAP (LIVE)

J1

K-Map for $J1 = Q0 \cdot \bar{X}$

Q1 \ Q0X	00	01	11	10
0	0	0	0	1
1	d	d	d	d

K-Map

d = don't care - colors mark the chosen prime implicants

OUTPUT 2: SEQUENTIAL CIRCUIT DIAGRAM

Circuit diagram is generated based on simplified equations. Q / Q' rails are driven by the flip-flop outputs.

1140503 張昱謙

GENERATE

EXPORT REPORT

圖 1 系統整體介面 (三欄式佈局，底部顯示作者學號姓名)

二、系統功能總覽

2.1 輸入區 (左欄)

- **Model Type**：選擇 Mealy 或 Moore 模型。
- **Flip-Flop Type**：選擇 JK、T 或 D 正反器。
- **State Table Input**：可自由命名輸入 / 輸出變數，並以可編輯的表格輸入狀態轉移，支援新增列、刪除列、清空、載入範例等操作。

2.2 運算結果區 (中欄)

- **State Transition Diagram**：將狀態表即時轉換為泡泡圖，圓圈代表狀態、箭頭代表轉換條件，不需按下 GENERATE 即時更新。
- **Flip-Flop Input Equations**：顯示化簡後的正反器輸入方程式與輸出方程式，並列出狀態編碼。
- **K-Map (Live)**：可下拉切換任一函數，以格雷碼排列並用顏色標示化簡所選的質含項群組。

2.3 電路圖區 (右欄)

- **Sequential Circuit Diagram**：依化簡後的方程式自動繪製電路圖，包含正反器、AND / OR 邏輯閘與接線，邏輯閘數量與佈線皆隨方程式自動調整。
- 提供放大、縮小、全螢幕、下載 SVG 等工具按鈕。

2.4 底部操作列

- 左側顯示作者資訊 (學號 1140503、姓名 張昱謙)。
- 中央為 GENERATE 主按鈕；當輸入變更後按鈕會轉為琥珀色，提示需重新產生。
- 右側為 EXPORT REPORT (匯出純文字設計報告) 與 About。

三、理論背景

3.1 Mealy 與 Moore 模型

Mealy 模型的輸出為「現態」與「輸入」的函數，輸出可能在輸入改變的當下立即變化；Moore 模型的輸出僅為「現態」的函數，輸出只在狀態改變時變化。本系統在 Mealy 模式下，輸出方程式包含輸入變數；在 Moore 模式下，系統會檢查同一狀態的輸出是否一致，並把輸出標示在狀態泡泡內。

3.2 正反器激勵表 (Excitation Table)

「激勵」是指要讓正反器從現態 $Q(t)$ 轉移到次態 $Q(t+1)$ 所需的輸入值。三種正反器的激勵表如下 (d 表示 don't care)：

$Q(t) \rightarrow Q(t+1)$	J	K	$Q(t) \rightarrow Q(t+1)$	T	$Q(t) \rightarrow Q(t+1)$	D
$0 \rightarrow 0$	0	d	$0 \rightarrow 0$	0	$0 \rightarrow 0$	0
$0 \rightarrow 1$	1	d	$0 \rightarrow 1$	1	$0 \rightarrow 1$	1
$1 \rightarrow 0$	d	1	$1 \rightarrow 0$	1	$1 \rightarrow 0$	0
$1 \rightarrow 1$	d	0	$1 \rightarrow 1$	0	$1 \rightarrow 1$	1

表 1 JK / T / D 正反器激勵表

由此可知：D 正反器最單純，其激勵 $D = Q(t+1)$ ，也就是「次態值」本身；T 正反器的激勵為 $T = Q(t) \oplus Q(t+1)$ ；JK 正反器則因含有 don't care，化簡後常能得到最精簡的邏輯。

3.3 卡諾圖與 Quine–McCluskey 化簡法

卡諾圖 (Karnaugh Map) 適合人工化簡 2~4 個變數的布林函數；而本系統內部採用

Quine–McCluskey (QM) 演算法進行化簡，原因是 QM 為表格式、步驟明確、易於程式化，且能正確處理任意變數數量與 don't care。其流程為：(1) 將最小項與 don't care 反覆兩兩合併，找出所有「質含項 (Prime Implicants)」；(2) 以「必要質含項」優先，再用貪婪法選取最少數量的質含項，覆蓋所有必須為 1 的最小項。中欄的卡諾圖即把 QM 所選的質含項以不同顏色圈選標示，方便對照。

四、完整製作流程

本章依實際開發順序，完整說明本專題從零到部署上線的每一個步驟。

4.1 需求分析與規劃

先確認系統要解決的核心問題：把「狀態表 → 正反器輸入方程式 → 電路圖」的人工流程自動化。據此規劃三欄式版面（輸入 / 結果 / 電路圖），並列出必備功能：模型與正反器選擇、可編輯狀態表、化簡方程式、卡諾圖、電路圖與狀態圖。

4.2 技術選型

- **React 19**：以元件化方式管理 UI 與狀態，輸入改變即自動重繪。
- **Tailwind CSS v4**：以 utility class 快速打造一致的工程軟體風格。
- **Vite 6**：開發伺服器啟動快、支援熱更新（HMR），打包輕量。
- **Lucide React**：提供乾淨的線條圖示。
- 全部以純前端實作，化簡與佈線皆於瀏覽器端計算，無需後端伺服器。

4.3 開發環境建置

開發初期本機尚未安裝 Node.js。為了能立即預覽，先製作了一份免安裝的 standalone.html（透過 CDN 載入 React / Tailwind / Babel），可直接用瀏覽器開啟。之後再正式安裝 Node.js LTS 並建立 Vite 專案：

```
winget install OpenJS.NodeJS.LTS # Node.js 24 LTS
npm install #
npm run dev # (localhost:5173)
```

4.4 介面實作

依規劃建立三欄式版面與卡片元件，抽出共用的 Card、Radio、IconButton 等小元件，並以 Tailwind 的 lg 斷點實作 RWD：螢幕寬度不足時三欄自動改為單欄垂直排列。左欄輸入、中欄結果、右欄電路圖分別以淡紅、淡藍、淡綠邊框作視覺區分。

4.5 核心演算法實作

這是系統的心臟，依序完成下列步驟：

- **狀態指定**：依狀態出現順序給定二進位編碼（如 A=00、B=01、C=10），並自動決定所需正反器數量 $n = \lceil \log_2(\text{狀態數}) \rceil$ 。
- **建立激勵表**：逐列依現態與次態，按 JK / T / D 的激勵規則填入各正反器輸入的真值表；凡未出現的（狀態, 輸入）組合一律視為 don't care。
- **布林化簡**：以 Quine–McCluskey 演算法求出每個輸入函數的最簡「積之和（SOP）」。
- **輸出方程式**：Mealy 由現態與輸入決定；Moore 僅由現態決定，並檢查輸出一致性。

4.6 視覺化實作

- **卡諾圖**：以格雷碼排列產生 2~4 變數的網格，並把化簡所選的質含項以顏色圈選。

- **電路圖**：依方程式的每個乘積項自動佈置 AND 閘，再以 OR 閘合併，並把輸入變數拉成左側匯流排、正反器輸出回授，全部以 SVG 動態繪製、可縮放與下載。
- **狀態轉換圖**：把狀態排成圓形佈局，依轉移畫出箭頭，自我迴圈與雙向轉換會自動彎曲避開，初始狀態加上起始箭頭。

4.7 測試與驗證

每完成一項功能，便在瀏覽器預覽中以實際操作驗證：切換模型 / 正反器、編輯狀態表、按下 GENERATE，並核對輸出方程式是否與手算結果一致（如後述第六章範例）。也測試了錯誤情況，例如空狀態、未定義的次態、重複的（狀態, 輸入）列、Moore 模型輸出不一致等，系統皆會顯示對應的錯誤或警告。

4.8 版本控制與部署

以 Git 進行版本控制，並推送至 GitHub。專案附帶 GitHub Actions 工作流程，每次 push 到 main 分支即自動建置（npm run build）並部署到 GitHub Pages：

```
git add -A
git commit -m "... "
git push # Actions
```

部署完成後，網站即可於下列網址公開使用：

<https://doremi0614.github.io/logic/>

4.9 後續功能擴充

在基本系統完成並上線後，又陸續加入三項擴充：(1) 即時**狀態轉換圖**；(2) 新增 **D 型正反器**（D = 次態值，激勵最單純）；(3) 將底部的 Settings 改為顯示作者**學號姓名**（1140503 張昱謙）。

五、系統架構與技術細節

5.1 檔案結構

```
logic/
├─ index.html #
├─ src/
│  ├─ main.jsx # React
│  ├─ App.jsx #   UI +
│  └─ index.css # Tailwind
├─ standalone.html #   CDN   App.jsx
├─ vite.config.js # Vite
├─ package.json #
└─ .github/workflows/deploy.yml # GitHub Pages
```

5.2 資料流

使用者輸入（狀態表、模型、正反器）儲存在 React 的 state 中。按下 GENERATE 時呼叫核心函數 `designCircuit()`，回傳狀態編碼、化簡後方程式與各函數的真值表；中欄與右欄的卡諾圖、電路圖元件再依此結果繪製。狀態轉換圖則直接讀取狀態表，因此不需按 GENERATE 即可即時更新。當輸入與上次產生的內容不同時，系統會標記為 `stale` 並提示重新產生。

5.3 關鍵技術點

- Quine–McCluskey 化簡同時支援 don't care，能對應正反器激勵表中的無關項。
- 電路圖以遞迴方式依乘積項數量計算 AND / OR 閘的位置與接線座標，達成自動佈線。
- 所有圖形皆為 SVG，可無損縮放並下載，方便放入報告或投影片。

六、操作說明與實作範例

6.1 操作步驟

- 於左欄選擇模型 (Mealy / Moore) 與正反器 (JK / T / D) 。
- 在 State Table 輸入狀態轉移；可按「Load Example」載入內建範例。
- 按下 GENERATE，中欄與右欄即顯示方程式、卡諾圖與電路圖。
- 需要時可下載電路圖 / 狀態圖 SVG，或按 EXPORT REPORT 匯出文字報告。

6.2 範例：以內建狀態表驗證

以系統內建範例 (Mealy 模型、JK 正反器) 為例，其狀態表如下：

現態 (Present)	X	次態 (Next)	Z
A	0	A	0
A	1	B	0
B	0	C	0
B	1	B	1
C	0	A	1
C	1	B	0

系統自動指定狀態編碼 **A=00**、**B=01**、**C=10** (狀態變數 Q_1Q_0)，經激勵表與 Quine–McCluskey 化簡後，得到下列結果：

$$J_1 = Q_0 \cdot X' \quad K_1 = 1 \quad J_0 = X \quad K_0 = X'$$
$$Z = Q_0 \cdot X + Q_1 \cdot X' \quad (\text{Mealy 輸出})$$

(註：X' 表示 X 的反相 NOT X。) 此結果與人工查激勵表、填卡諾圖化簡所得完全一致，驗證系統正確。

七、成果展示

下列為上述範例由系統自動產生的狀態轉換圖與循序電路圖。

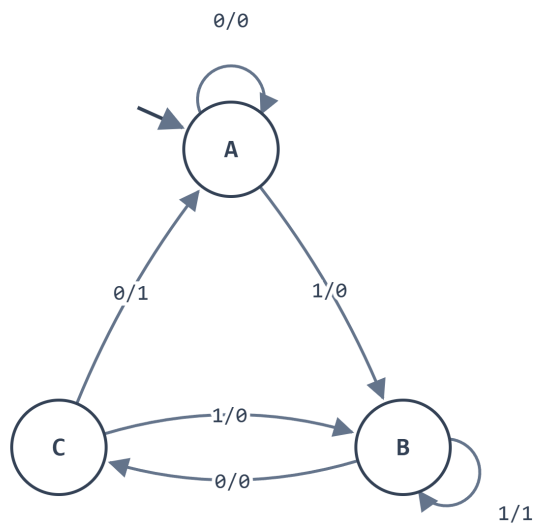


圖 2 狀態轉換圖：圓圈為狀態、箭頭標示「輸入 / 輸出」，含自我迴圈與起始箭頭

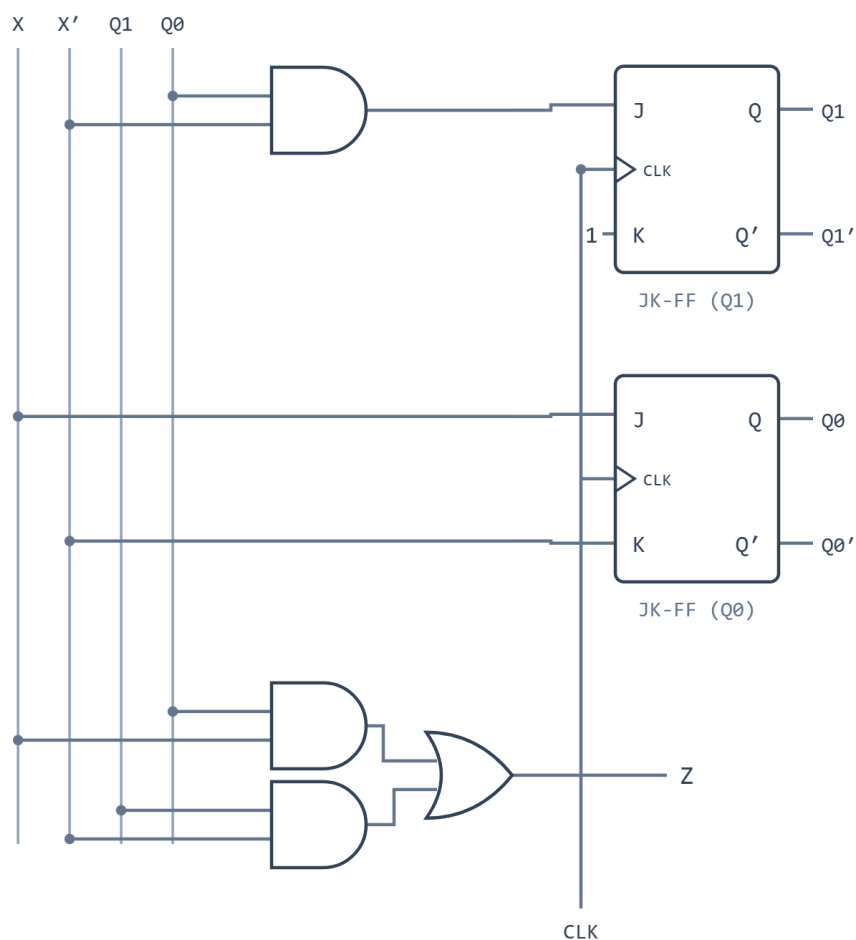


圖 3 循序電路圖：兩個 JK 正反器、自動佈置的 AND / OR 閘與接線，右側輸出 Z

系統已部署上線，任何人皆可於瀏覽器開啟下列網址實際操作：<https://doremi0614.github.io/logic/>

八、開發過程遭遇的問題與解決

8.1 本機尚未安裝 Node.js

問題：開發初期環境沒有 Node.js / npm，無法立即建立 Vite 專案預覽。

解決：先以 CDN 製作免安裝的 standalone.html 快速預覽；之後再用 winget 安裝 Node.js LTS 建立正式專案。

8.2 standalone.html 中文亂碼

問題：由程式自動產生 standalone.html 時，中文與 `、` 等符號出現亂碼。

解決：改以明確的 UTF-8 (無 BOM) 讀寫檔案，確保特殊字元正確保存。

8.3 git push 被拒 (fast-forward)

問題：在 GitHub 網頁上操作後，遠端多出本機沒有的提交，導致 push 被拒。

解決：先 git pull 整合遠端變更後再 push。

8.4 兩個部署工作流程衝突

問題：從 GitHub 頁面套用了 Static HTML 範本，產生的 static.yml 會把未建置的原始碼直接上傳，與自訂的 deploy.yml (會先 build) 互相競爭，可能導致網站壞掉。

解決：刪除多餘的 static.yml，只保留會先建置 Vite 專案的 deploy.yml。

8.5 Tailwind v4 色彩與截圖工具相容性

問題：Tailwind v4 預設使用 oklch() 色彩，傳統截圖函式庫無法解析。

解決：改用支援 oklch 的 html2canvas-pro 擷取介面，並把 SVG 圖以瀏覽器 canvas 轉為高解析 PNG。

九、結論與心得

本專題完成了一套可實際運作的「循序電路設計自動化系統」，把課堂上繁瑣的人工設計流程——狀態指定、激勵表、卡諾圖化簡、繪製電路圖——整合為輸入狀態表即可一鍵完成，並支援 Mealy / Moore 模型與 JK / T / D 三種正反器。

在製作過程中，我把課本的理論（激勵表、Quine–McCluskey 化簡）實際寫成程式，對演算法的理解比單純做習題更深刻；也學到完整的前端工程流程：從環境建置、元件化開發、瀏覽器驗證，到使用 Git 與 GitHub Actions 自動部署上線。過程中遇到的種種問題與排解，更是一次完整的工程實戰經驗。

未來可再擴充的方向包括：支援多個輸入 / 輸出變數、自動狀態化簡（state minimization）、提供電路時序模擬，以及匯出標準的 Verilog / VHDL 程式碼。

十、參考資料

- M. Morris Mano, Michael D. Ciletti, Digital Design.
- Quine–McCluskey algorithm — 布林函數最小化演算法。
- React 官方文件 <https://react.dev/>
- Tailwind CSS 官方文件 <https://tailwindcss.com/>
- Vite 官方文件 <https://vite.dev/>
- 專案原始碼 <https://github.com/doremi0614/logic>
- 線上展示 <https://doremi0614.github.io/logic/>